1. Use Case Description

Helix Medical Centre (HMC) is a fictitious tertiary hospital and genomics research facility that offers clinical pharmacogenomics testing to patients across five clinical sites. Oncologists, cardiologists, and clinical pharmacists at HMC use PGx testing to guide drug selection and dosing decisions for patients with cancer, cardiovascular disease, and psychiatric conditions.

HMC has commissioned you, a bioinformatics data engineer to design and prototype a MongoDB-based PGx Registry to replace their disorganized collection of lab spreadsheets and text reports. The new system must:

- Store and manage records for registered patients, gene panel tests, individual genomic variants catalogued per panel, and drug response observations recorded for patients carrying specific variants.
- Support parameterized, generalizable analytical queries covering drug safety monitoring, variant prevalence, panel utilization, and patient-level genomic profiling.
- Expose the data through a well-structured RESTful API (FastAPI) that accepts dynamic inputs and returns structured JSON responses.
- Provide an interactive portal where clinicians and researchers can search patients, browse panels, inspect variant catalogues, and view drug response summaries, all driven by the API.

2. Data Description

The PGx Registry dataset consists of four primary entities that map directly to four MongoDB collections. The tables below describe each entity's attributes, data types, and value ranges. Use these as your authoritative field-level reference for schema design and data generation. However, this section describes entities in isolation. How these entities relate to one another: the connections, cardinalities, and constraints between them, is conveyed in the client requirements recording. Both sources are required to produce a correct schema design.

2.1 Entity: Patients

Represents a registered patient who has undergone at least one gene panel test at HMC. Patient records store demographic, clinical, and panel-ordering information.

Attribute	Type	Values / Range
patient_id	String	Format: PT-XXXXXX (e.g. PT-001001). Unique identifier.
name	String	Synthetic full name (Firstname Lastname).
date_of_birth	Date	ISO 8601. Range: 1945-01-01 to 2005-12-31.
gender	String	Enum: Male | Female | Non-binary | Prefer not to say.
ethnicity	String	Enum: Malay | Chinese | Indian | Caucasian | African | Hispanic | Other.

blood_type	String	Enum: A+ | A- | B+ | B- | AB+ | AB- | O+ | O-.
primary_condition	Object	Embedded: { icd10_code: String, description: String, diagnosed_on: Date }. ICD-10 compliant.
comorbidities	Array	Array of Strings (ICD-10 code + label). 0–4 entries. E.g. 'E11.9 - Type 2 diabetes mellitus'.
site_id	String	Clinic site. Enum: SITE-01 to SITE-05.
ordered_panels	Array	Array of panel_id Strings referencing the gene_panels collection. 1–3 entries.
contact_info	Object	Embedded: { email: String, phone: String, emergency_contact: String }. Synthetic.
created_at	Date	ISO 8601 record creation timestamp.

2.2 Entity: Gene Panels

Represents a diagnostic gene panel test: a curated set of genes analysed together for a specific clinical purpose. Each panel has a defined catalogue of variants it is designed to detect.

Attribute	Type	Values / Range
panel_id	String	Format: PNL-XXXXXX (e.g. PNL-000101). Unique identifier.
panel_name	String	Descriptive name. E.g. 'Comprehensive Oncology Panel v3', 'CYP Metaboliser Panel v2'.
panel_type	String	Enum: Oncology | Pharmacogenomics | Cardiology | Rare Disease | Infectious Disease.
version	String	Format: vX.Y (e.g. v1.0, v2.3, v3.1).
status	String	Enum: Active | Deprecated | Under Review.
target_genes	Array	Array of HGNC gene symbols the panel screens. E.g. ['BRCA1','BRCA2','TP53','EGFR']. 5–30 entries.
turnaround_days	Integer	Typical result turnaround. Range: 3–21 days.
ordering_sites	Array	Array of site_id Strings where this panel is available. 1–5 entries.
manufacturer	String	E.g. 'HMC In-house', 'Illumina', 'Foundation Medicine', 'Myriad Genetics'.
regulatory_clearance	String	Enum: CE-IVD | FDA-cleared | Research Use Only.
clinical_utility	String	Short statement of clinical purpose. E.g. 'Guide targeted therapy selection in solid tumours'.
created_at	Date	ISO 8601 timestamp.

2.3 Entity: Variants

Represents a single genomic variant catalogued within a gene panel. Each variant entry describes a specific mutation or polymorphism that the panel is designed to detect, its clinical significance, and the condition it is associated with. Variants are defined at the panel level that represent what the panel can find, not what was found in any individual patient.

Attribute	Type	Values / Range
variant_id	String	Format: VAR-XXXXXX (e.g. VAR-000101). Unique identifier.
panel_id	String	Reference to the gene_panels collection. The panel this variant belongs to.
gene_symbol	String	HGNC gene symbol. E.g. BRCA1, CYP2D6, EGFR, DPYD, TPMT, VKORC1, CYP2C19.
hgvs_notation	String	HGVS coding notation. E.g. c.1799T>A, c.3140A>G, c.2T>C. Unique per variant.
rsid	String	dbSNP reference SNP ID. E.g. rs113488022. Null if novel/unpublished variant.
chromosome	String	Enum: 1–22 | X | Y.
position	Integer	GRCh38 genomic coordinate. Range: 1 – 248,956,422.
ref_allele	String	Reference allele. Single nucleotide or short sequence. E.g. T, AG, CATG.
alt_allele	String	Alternate allele. Single nucleotide or short sequence. E.g. A, G, del.
variant_type	String	Enum: SNV | Indel | CNV | Fusion | Splice variant.
zygosity_expected	String	Enum: Heterozygous | Homozygous | Hemizygous.
clinical_significance	String	Enum: Pathogenic | Likely pathogenic | Variant of Uncertain Significance (VUS) | Likely benign | Benign. Follows ClinVar classification.
associated_condition	String	Primary condition linked to this variant. E.g. 'Hereditary breast and ovarian cancer', 'Poor CYP2D6 metabolism', 'Non-small cell lung cancer'.
inheritance_pattern	String	Enum: Autosomal dominant | Autosomal recessive | X-linked | Mitochondrial | Not applicable.
evidence_level	String	AMP/ASCO/CAP tiered evidence. Enum: 1A | 1B | 2A | 2B | 3 | 4. 1A = highest clinical evidence.
created_at	Date	ISO 8601 timestamp.

2.4 Entity: Drug Responses

Records a single observed drug response associated with a patient who carries a specific variant. Each drug response links a patient, the panel through which the variant was discovered, and the specific variant that is clinically relevant to the drug interaction. This is the entity through which genomic findings are translated into clinical pharmacology outcomes.

Attribute	Type	Values / Range
response_id	String	Format: DR-XXXXXXXX (e.g. DR-00100001). Unique identifier.
patient_id	String	Reference to the patients collection.
panel_id	String	Reference to the gene_panels collection. The panel through which the variant was identified.
variant_id	String	Reference to the variants collection. The variant driving this drug response.
drug_name	String	Generic drug name. E.g. Tamoxifen, Imatinib, Warfarin, Clopidogrel, Codeine, Fluorouracil.
drug_class	String	Enum: Chemotherapy | Targeted therapy | Anticoagulant | Antidepressant | Analgesic | Immunosuppressant | Other.
response_type	String	Enum: Efficacy | Toxicity | Dosing. Characterises what aspect of the drug response was affected by the variant.
outcome	String	Enum: Normal metaboliser | Poor metaboliser | Ultra-rapid metaboliser | Intermediate metaboliser | Increased sensitivity | Reduced efficacy | Adverse reaction | Standard response.
phenotype_observed	String	Free-text clinical observation. E.g. 'Grade 3 myelosuppression', 'Subtherapeutic plasma levels', 'Bleeding event'.
ctcae_grade	Integer	CTCAE v5.0 toxicity grade: 1–5. Null if response_type is Efficacy or Dosing only.
recommendation	String	Clinical action. E.g. 'Reduce dose by 50%', 'Avoid use — select alternative', 'Standard dosing applies', 'Increase monitoring frequency'.
evidence_source	String	Enum: FDA label | PharmGKB | CPIC guideline | DPWG guideline | Institutional protocol.
reported_by	String	Enum: Clinical pharmacist | Oncologist | Cardiologist | Geneticist | Sponsor.
reported_on	Date	ISO 8601. Date the response was formally recorded. Range: 2019-01-01 to 2024-12-31.
created_at	Date	ISO 8601 record creation timestamp.

3. Data Relationships

The four entities in this system do not exist in isolation. They are interconnected in ways that must drive your MongoDB schema design. The paragraphs below describe each major relationship in plain language. Use them alongside the client requirements recording, which provides the additional detail including the cardinality, mandatory constraints, and design implications that you are expected to extract and apply independently.

- A Gene Panel is the central catalogue entity in this system. Every panel has a defined catalogue of Variants — specific genomic changes that the panel is validated to detect. Each variant in the system belongs to exactly one panel; a variant without a panel context has no clinical meaning in this registry. A single panel may contain many variants across the genes it screens, while each variant is scoped to only that one panel. This scoping is important: the clinical evidence and interpretation associated with a variant are tied to the panel it belongs to.
- A Patient is an individual who has been referred for at least one gene panel test. Each patient is registered at a single clinical site, which does not change. A patient carries a record of the panels they have been tested with, and a panel in turn is associated with many patients who have ordered it. This means the relationship between patients and panels runs in both directions — it is not a simple one-way association, and this bidirectionality has direct implications for how the linkage should be stored in MongoDB.
- A Drug Response is the most relationship-dense entity in the system. Every drug response must be associated with the patient who experienced it, the gene panel through which the relevant variant was identified, and the specific variant that is driving the drug interaction. All three of these associations are mandatory — a record missing any one of them is considered incomplete and clinically meaningless. A single patient may have many drug response records across different panels and variants, and a single variant may be associated with many drug responses across many different patients.
- A drug response record does not store the gene symbol directly. To find all drug responses associated with a specific gene — for example, all responses linked to CYP2D6 — you must first identify all variants in the system where the gene symbol matches, collect their variant IDs, and then retrieve all drug response records that reference those variant IDs. This two-step path — from drug response to gene, via variant — is the most important query pattern in the system. Understanding this indirect linkage is essential for designing both your queries and your API endpoints correctly.

Note: Please listen to the recording for more specific information

4. Analytical Requirements

This section defines ten analytical operations the system must support. Each operation represents a real data need from the client. For every analytical requirement, you are expected to complete three things in sequence: design a MongoDB query or aggregation pipeline that fulfils it, implement that query as a generalizable FastAPI endpoint that accepts appropriate parameters so it works for any valid input (not just a hardcoded example), and connect the endpoint to the data portal so users can invoke it interactively.

The data portal is the front-end of this system. It should provide a coherent interface where a user (a clinician, pharmacist, or researcher) can navigate the data, apply filters, and view results and charts without any knowledge of the underlying database or API. Every analytical requirement below should be reachable through the portal in some form, whether as a search filter, a data table, or a visualization.

Note: The quality of your API design — how you name endpoints, structure parameters, and handle varied inputs — is assessed independently from query correctness. Refer to the naming conventions in Section 6 (D5) before designing your endpoints.

#	Operation	Analytical Requirement
AR1	Filter gene panels by type and/or status	Retrieve a list of gene panels filtered by one or more attributes such as panel type, status, or manufacturer.
AR2	Retrieve all patients for a specific gene panel	Given a panel, retrieve the demographic details of all patients who ordered that panel, with the ability to narrow results further by patient attributes.
AR3	Search patients by demographic or clinical criteria	Search across the patient population using combinations of demographic and clinical attributes such as gender, ethnicity, site, or primary condition.
AR4	Retrieve all drug responses for a patient	Given a patient, retrieve all drug response records across all panels and variants, with the ability to filter by response type or other attributes.
AR5	Drug response summary grouped by drug class	Aggregate drug response records grouped by drug class, returning counts and the proportion classified as toxicity for each class.
AR6	Variant catalogue for a gene panel	Given a panel, retrieve its full variant catalogue with the ability to filter by clinical significance or evidence level.
AR7	Drug response outcome matrix for a variant	For a given variant, produce a cross-tabulation of all associated drug responses by outcome category and response type.
AR8	Patient comorbidity and drug response burden	Identify patients whose comorbidity count exceeds a given threshold and return their total and toxicity drug response counts.
AR9	Variants and responses by gene symbol	Retrieve all variants associated with a specified gene symbol and for each return the count of drug response records linked to it.

AR10	Monthly drug response trend over time	Produce a time-series of drug response records grouped by year and month, with the ability to scope results to a specific panel or response type.

Important: Each analytical requirement must result in a working MongoDB query demonstrated with a real example from your dataset, a generalizable API endpoint that accepts parameters rather than hardcoded values, and a reachable feature in the data portal. Implementations that only work for one fixed input will not receive full marks.

5. Deliverables

Students must submit all eight deliverables below. The table describes what each deliverable is and the expected format. How each is designed and implemented is your responsibility.

#	Deliverable	Description	Format
D1	MongoDB Schema Design	The MongoDB structure design (in JSON schema) of the solution	.json
D2	Data Ingestion Code	pymongo ingestion script that connects to the MongoDB, transforming all the raw data and store in MongoDB according to the MongoDB schema of your solution. Must include a screenshot or console output showing at least 3 sample documents after ingestion.	Python (.py / .ipynb) + Screenshot (PNG/PDF)
D3	Database Backup	mongodump archive of the populated database.	.tar.gz or .zip
D4	Queries & Results (10)	MongoDB query code + results for all 10 ARs.	Python (.py / .ipynb) + results (PDF)
D5	FastAPI Implementation	All API endpoints implemented and tested.	Python (.py)
D6	Data Portal	Read-only interactive portal consuming the API. Using Streamlit or Gradio.	Python (.py)
D7	Technical Report	Written report following the outline in Section 7.	PDF
D8	System Demonstration Video	Screen-recorded walkthrough. Strict duration: 8–10 minutes.	.mp4 or .mov

Note: D6 is a read-only data browsing and analytics portal. No data entry, editing, or deletion features are required. All data enters the system through D2; the portal surfaces that data through the API only.

Additional Notes for API Naming & Design Conventions

Your FastAPI implementation must follow these conventions. They will be used as assessment criteria for D5.

Convention	Requirement
URL format	Lowercase, hyphen-separated path segments. E.g. /api/gene-panels.
Resource naming	Plural nouns for collections. E.g. /api/panels, /api/patients.
Path parameters	Use for required resource identifiers. E.g. /api/panels/{panel_id}.
Query parameters	Use for optional filters and pagination. E.g. ?panel_type=Oncology&page=1&limit=20.
HTTP methods	GET only (read-only API).
Response envelope	All list responses: { total: int, page: int, limit: int, data: [...] }.
Error responses	HTTPException with appropriate status codes: 404, 422, 500.
Auto-docs	FastAPI /docs (Swagger UI) must be functional.

Additional Notes for Demonstration Video

A single continuous screen recording, 8–10 minutes, with audio narration throughout. No editing required. The video must cover the five segments below in order.

#	Segment	What to Show & Say
1	**Database proof**	All four collections in Compass or shell with document counts. Navigate into at least one document per collection and narrate key fields and referential consistency.
2	**Live API via Swagger UI**	Execute at least four endpoints live: one path parameter endpoint, one multi-filter endpoint, one analytics endpoint. Narrate the request and what the response represents.
3	**Portal walkthrough**	Demonstrate all five portal features. For each, interact with at least one filter and state which API endpoint is being called.
4	**Query explanation**	Show the pipeline code and result for any two of your 10 queries. Explain each aggregation stage.
5	**Challenges reflection**	Describe one specific technical challenge encountered, what you tried, and how you resolved it.

6. Technical Report Outline (D7)

The written report must follow the structure below.

Section	Section Title	What to Cover
1	Introduction	Background and motivation for the system. State your objectives and justify the choice of MongoDB for this domain.
2	System Architecture	Overview of the three-layer system (database, API, portal) and how data flows between them. Include a diagram.
3	Database Schema Design	Present and justify your schema for each collection. Explain your embed vs. reference decisions and how the schema supports the analytical requirements.
4	Data Ingestion	Describe how sample data was generated and loaded. Show at least one sample document per collection and discuss how referential consistency was maintained.
5	Query Design (10 ARs)	For each AR: state what it retrieves, show the MongoDB pipeline, show the result, and briefly explain what the output means.
6	API Design & Implementation	Describe your endpoint structure, naming decisions, and how each endpoint generalizes the corresponding query. Discuss how parameters were handled.
7	Data Portal	Describe the portal and its technology stack. Include screenshots of the key features and explain how they connect to the API.
8	Challenges & Solutions	Describe at least three real technical problems you encountered and how you resolved them. Must be specific to your implementation, not generic.
9	Conclusion	Summarize what was achieved and propose meaningful future improvements.
Ref	References	Cite all tools, libraries, standards, and sources used (if any).

7. Marking Rubric

Component	Marks	Key Assessment Criteria
D1 — Schema Design	10	Correct JSON Schema syntax; required/optional justification; type constraints; nested definitions.
D2 — Ingestion Code & Output	10	Code quality; genomically plausible data; referential consistency; sample documents match schema.
D3 — Database Backup	5	Valid mongodump archive that restores cleanly; document counts match D2.
D4 — Queries & Results	20	Correctness of all 10 pipelines; non-empty results; quality of explanation.
D5 — FastAPI Implementation	25	All 10 endpoints; naming conventions; parameterization; error handling; Swagger docs functional.
D6 — Data Portal	10	Coverage of 5 features; correct API integration; dynamic chart updates; color-coded response table.
D7 — Technical Report	10	Structure; schema design justification; API design discussion;
D8 — Demonstration Video	10	All 5 segments in order; live API calls; portal features shown with interactions; pipeline explained; challenge reflection specific and concrete. Duration within 8–10 minutes.
TOTAL	**100**	**—**

8. Submission Guidelines

Submit a single ZIP archive named: **GROUPNAME_MiniProject.zip**

Required folder structure:

```
GROUPNAME_MiniProject/
  D1_schemas/
  D2_ingestion/
  D3_backup/
  D4_queries/
  D5_api/
  D6_portal/
  D7_report/
  D8_video/
  AI_Declaration/
  README.md # Setup & run instructions for API and portal
```

9. Academic Integrity

All submitted work must be your own. You are expected to understand, annotate, and be able to explain every line of your submission. Identical query code or schema designs between students within or across groups will be treated as academic misconduct.

The use of generative AI tools such as ChatGPT, Claude, Copilot, or similar is permitted as a learning aid, subject to the following conditions:

- You may use AI to assist with understanding concepts, debugging, and generating boilerplate code. You may not use AI to generate your schema design, analytical queries, or written report in full.
- You must be able to explain and justify every part of your submission independently.
- If you used any AI tool at any point during this project, you must attach the relevant conversation log(s) as a PDF in an AI Declaration folder included in your submission ZIP. This is mandatory — omitting it when AI was used is a breach of academic integrity.

The AI Declaration folder should clearly label which deliverable each conversation relates to, for example: Schema Design, Query, Report Section.

— End of Assignment Brief —